# CACHE MEMORY

In this chapter, we will discuss some of the basics of *cache memory* and briefly discuss its performance. To arrive at the need for cache memory, we will start with an overview of different types of computer memory.

## 1. Computer memory: an overview

Since the advent of the very first computer systems, it has become apparent that the speed of computer memory is usually insufficient to meet the speed at which the processor requests information from the memory. Since then, this discrepancy has only increased. The type of memory that does meet these requirements is very expensive and is very limited in size. Consequently, it seems that one must be limited to systems with a very scarce amount of memory or to slow processors. To get around these limitations, computer memory has been given a hierarchical structure.

Memory is typically divided into internal and external memory. With internal memory we think of the *registers* present in the processor, the cache memory and the *main memory* or RAM memory. This memory is mostly *volatile*, meaning that it loses the information it carries as soon as there is no more electrical current present. This is in contrast to external memory which consists of external devices such as *disks*, *flash* memory and *tapes*.

### Memory Hierarchy

As indicated in previous section, there are many types of memory. As a result, most of today's computer systems have a whole range of types of computer memory. Typical usage is well represented by the following memory hierarchy (see Figure 7). As we descend in this hierarchy, the following things happen:

1. The price per bit decreases,
2. The size of memory increases,
3. The access time (that is, time to access the memory) increases, and
4. The frequency with which the processor requests data from memory decreases.

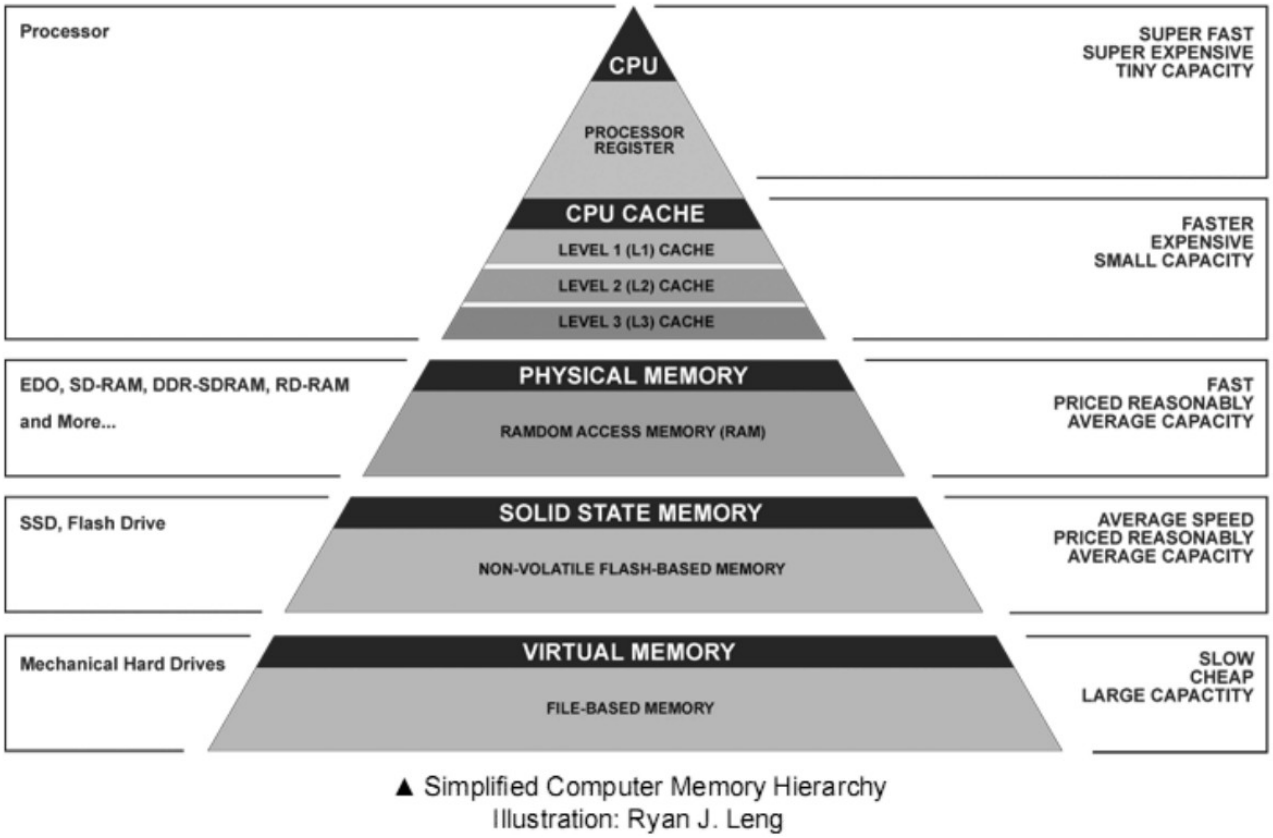
The latter is the key to the success of memory organization in our current computer systems. Unless in the majority of cases the retrieved data is also present in fast memory, there is little point in having this expensive memory. Fortunately, there is such a thing as *locality of reference*, which means that a particular instruction or data is frequently requested in short succession (think loops) or that instructions or data near the current instruction/data will also be requested with a fairly high probability in the near future (think often sequentially, or arrays, etc.). We will return to this idea frequently in the remainder of this and other chapters.

For cache size, we have to think in terms of a Kbyte to a few Mbytes. For memory, we quickly go up to a few Gbytes, while disks today can hold a few Tbytes. The access time for an on-chip cache (a few nanoseconds) is up to 40 times faster than the access time of main memory, while a main memory access is often 10000 times faster than a disk access (a few milliseconds). We will return to this later in this chapter.

## 2. Cache memory organization.

Cache memory is usually the fastest memory that is not itself part of the processor (although it is usually located on the processor's chip). In addition to the very low access time that this type of memory can achieve (e.g., 4 cycles), it is also important that the connection between the cache and the processor is very fast. If the cache memory would have to share a bus with the main memory,

the I/O equipment, the network equipment, etc. then there is little profit left of the low access time. Consequently, a separate, very fast bus is used to connect the processor and the cache.




▲ Simplified Computer Memory Hierarchy
Illustration: Ryan J. Leng

Fig. 7: Memory hierarchy.

The main memory will then be connected to the cache via a slower bus, which may well be shared with other devices (see Figure 8).

Another distinction between the two buses is that the unit with which the processor and cache exchange data corresponds to the previously mentioned *word size*, while between the memory and the cache one will exchange larger blocks (consisting of K words). This is also logical given the so-called locality of reference property, as well as the fact that the slower bus is shared and one should therefore exchange a bit more data when the bus is available.

The content of the cache contains a (small) subset of the information in the main memory. Let the main memory contain a total of $2^n$ addressable words (so the memory size is equal to $2^n$ times the word size). Then this memory is partitioned into a collection of blocks consisting of $K$ consecutive words: the first block contains the first $K$ words, the second the next $K$, and so on. The main memory thus contains exactly $M = 2n /K$ of such blocks. A cache consisting of *C lines* accommodates exactly $C << M$ of these blocks (each consisting of K words). In addition to these $K$ words, each line also contains a *tag*, which is needed to identify which block from main memory is present on this line in the cache. The cache also contains a single bit per line that indicates whether the line is in use and thus contains meaningful data (as well as an UPDATE bit, etc.).

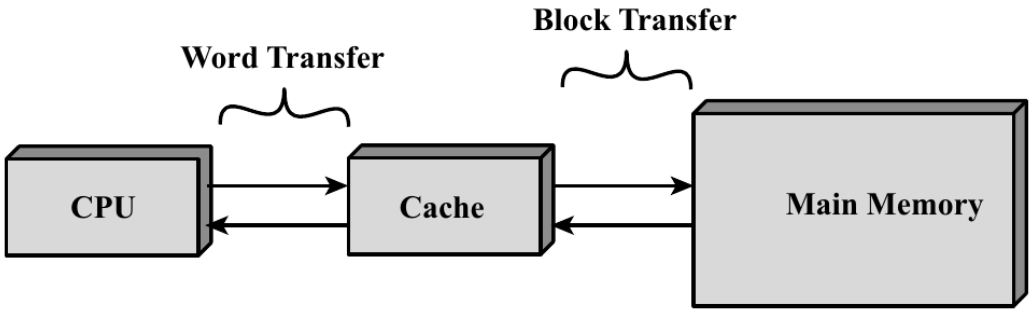



Fig. 8: Cache connections [Stallings 03].

When the processor wishes to read a particular word from memory, it will first consult the cache. How the cache controller, which receives this query (containing the address of the word), verifies this depends on how the cache is designed. Further in this chapter, we will discuss three such designs. If the requested word is found, we refer to this as a *hit*, then the cache will pass the requested information directly to the processor. In this case, the slower main memory is not consulted and we do not need to use the shared, less fast bus. When the block containing the requested word is missing, the cache will retrieve the block containing this information from main memory and store it in one of the cache lines for possible later reuse.

For efficient operation of the cache structure, it is vital that the probability *H* that a requested word is in the cache is high. Without the aforementioned locality of reference, this probability, called the *hit ratio*, is only equal to $C/M \approx 0$ and thus we almost always have to fall back to main memory. Of course, this probability *H* is also influenced by the choice of the line in which we store a new block in the cache. Further in this chapter we will discuss a number of strategies for this. In addition, *H* will also be a function of the number of lines *C* in the cache. The larger the cache, the more likely it is that a particular word is actually in the cache. However, the larger *C*, the more expensive (and slower) the cache becomes.

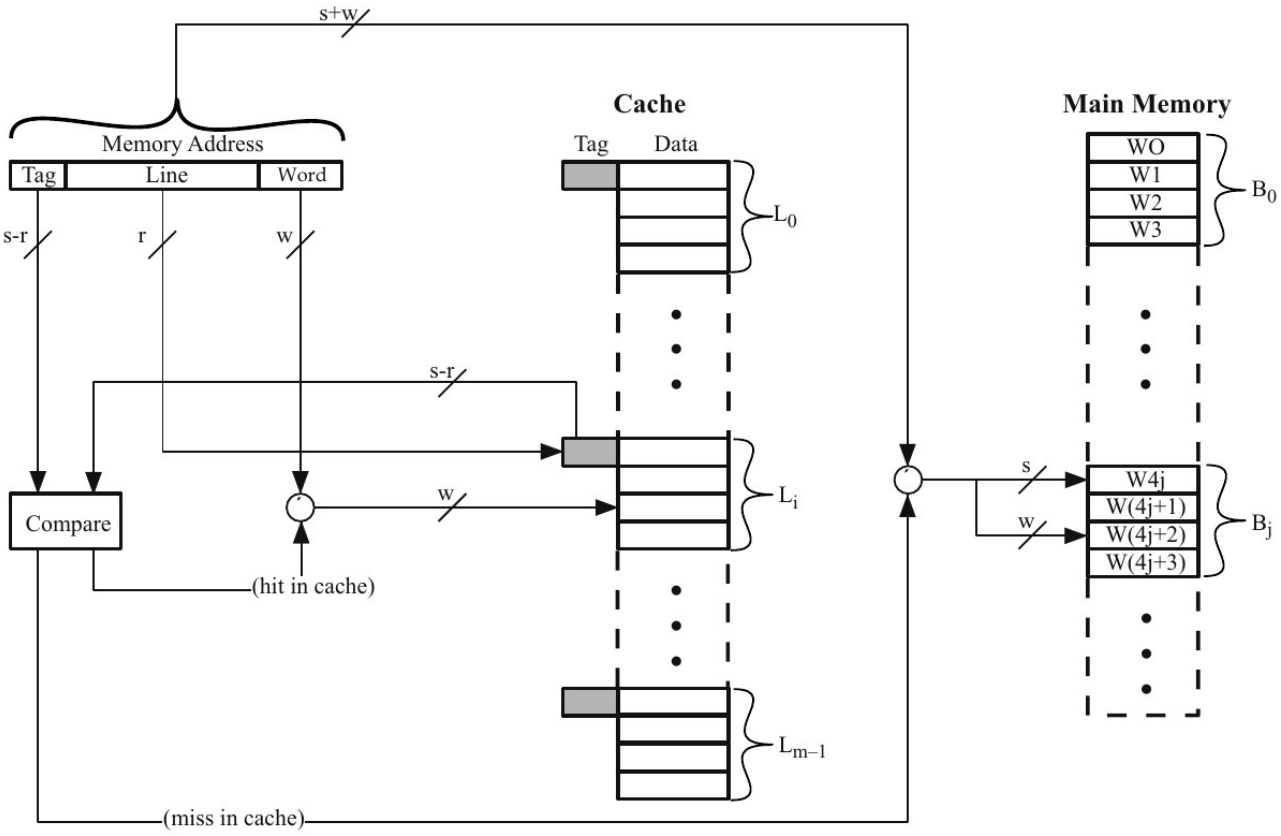



Fig. 9: Direct Mapping [Hwang 93], m = C

**2.1 Direct mapping.**

As we noted earlier, the number of lines in the cache $C = 2^r$ is significantly smaller than the number of blocks $M$ in main memory. The *mapping function* will indicate where to place block number $j$ in the cache. From a mathematical point of view, the name function is not well chosen, since most mappings allow a block to be written in different places (so there is a choice). However, the simplest mapping, called *direct mapping*, will map each block to a single line $i$ by the simple relationship

$$i = j \bmod 2r\ (= C) \qquad (1)$$

Suppose $K = 2^w$ , so each block (and each line in the cache) contains exactly $2^w$ words. Then the last $w$ bits, which we call the least significant, of an address will indicate the number of words in a block to which this address corresponds.

Now if we look at the address of a particular word, which consists of $a = s + w$ bits, then the remaining $s$ bits, i.e. the first ones, will form the number $j$ of the block. If we now wish to calculate the corresponding line number, it suffices to look at the $r$ least significant bits of the first $s$ bits. All addresses that thus have the same $r + w$ last bits are all mapped to the same line of the cache. The tag, which identifies which block is effectively in memory, thus corresponds exactly to the $s - r$ first bits of the address. Note that by this choice of mapping, we ensure that the blocks that are mapped to the same line are also quite far apart in memory (namely, exactly $2^{r+w}$ places).

The operation of the cache and the way the controller checks whether the requested word is in the cache is demonstrated on the example of Figure 9. First, the controller will determine the cache line by looking at the $r$ middle bits. Next, the tag of this line is compared with the tag of the requested address. If it matches then we find the requested word in the cache by looking at the w least significant bits of the address. Otherwise we need to consult the main memory.

So we can summarize the direct mapping method as follows:

- Address *length* $= a = s + w$ bits,
- Number of addressable words is equal to $2^a$ ,
- Block size = line size $K = 2^w$ words,
- Number of blocks in memory $M = 2^a / 2^w = 2^s$ ,
- Number of lines in the cache $C = 2^r$ ,
- Number of bits in the *tag* $= s - r$ bits.

Although this technique is very simple, it comes with a number of drawbacks. For example, if one needs data alternately contained in two blocks mapped to the same line, then one block will always overwrite the other, and so we have to fall back to main memory each time. This phenomenon is called *thrashing*. This is not very common since memory blocks that use the same cache line are quite far apart and we have the locality of reference phenomenon. However, when executing a simple loop where one block contains the instructions and the other happens to contain the data, this drawback can be very significant. Another example that at first glance realizes a good cache hit rate is shown in Figure 10. We assume that we have a 4 Kbyte cache with blocks of 32 bytes. A float has a size of 4 bytes, so each cache line contains 8 floats and we assume that both the variable $a$ and $b$ are not yet in the cache. It appears that we realize a hit rate of 87.5 percent, which is usually correct. However, when $a$ and $b$ are in memory right after each other then $a[i]$ is exactly 4 Kbyte away from $b[i]$ and we get a hit rate of 0 percent. Note that the same problem would occur even when the arrays $a$ and $b$ have a multiple of 1024 as their size.

```
float a[1024], b[1024];

for (i=0; i<1024; i++)
  sum += a[i]*b[i];
```

The generic system executes this loop as follows:

```
i   Operation Status In cache  Comment

0   load a[0] miss   a[0..7]   assume a[] was not cached yet
    load b[0] miss   b[0..7]   assume b[] was not cached yet
    t=a[0]*b[0]
    sum += t
1   load a[1] hit    a[0..7]   previous load brought it in
    load b[1] hit    b[0..7]   previous load brought it in
    t=a[1]*b[1]
    sum += t
    ..... etc .....
7   load a[7] hit     a[0..7]  previous load brought it in
    load b[7] hit     b[0..7]  previous load brought it in
    t=a[7]*b[7]
    sum += t
8   load a[8] miss    a[8..15] this line was not cached yet
    load b[8] miss    b[8..15] this line was not cached yet
    t=a[8]*b[8]
    sum += t
9   load a[9] hit     a[8..15] previous load brought it in
    load b[9] hit     b[8..15] previous load brought it in
    t=a[9]*b[9]
    sum += t
    .......
```

Fig. 10: Cache trashing example

**2.2 Associative and set-associative mapping.**

In the case of associative mapping, any memory block may be written to any possible line in the cache. Thus, the line is no longer determined by the *r* middle bits. Consequently, the thrashing phenomenon will no longer occur to the same extent. However, with associative mapping the tag has a length of s bits (instead of *s* - *r*). Since each block can now be anywhere in the cache, we must now remember the full block number, which corresponds exactly to the first s bits of the address.

If we wish to check whether a certain word is in the cache, we must compare the tag of the requested word with all the tags in the cache (see Figure 11). This will be done by comparing all the tags in parallel with the requested tag. We call this type of (cached) memory access *associative*, as opposed to the previous one which we call *direct access*. When the word is not in the cache, we will retrieve the block it is part of from main memory and store it in the cache. The place where we store this block, we can now choose freely unlike the direct mapping technique. Algorithms for this that attempt to maximize the hit ratio will be discussed further.

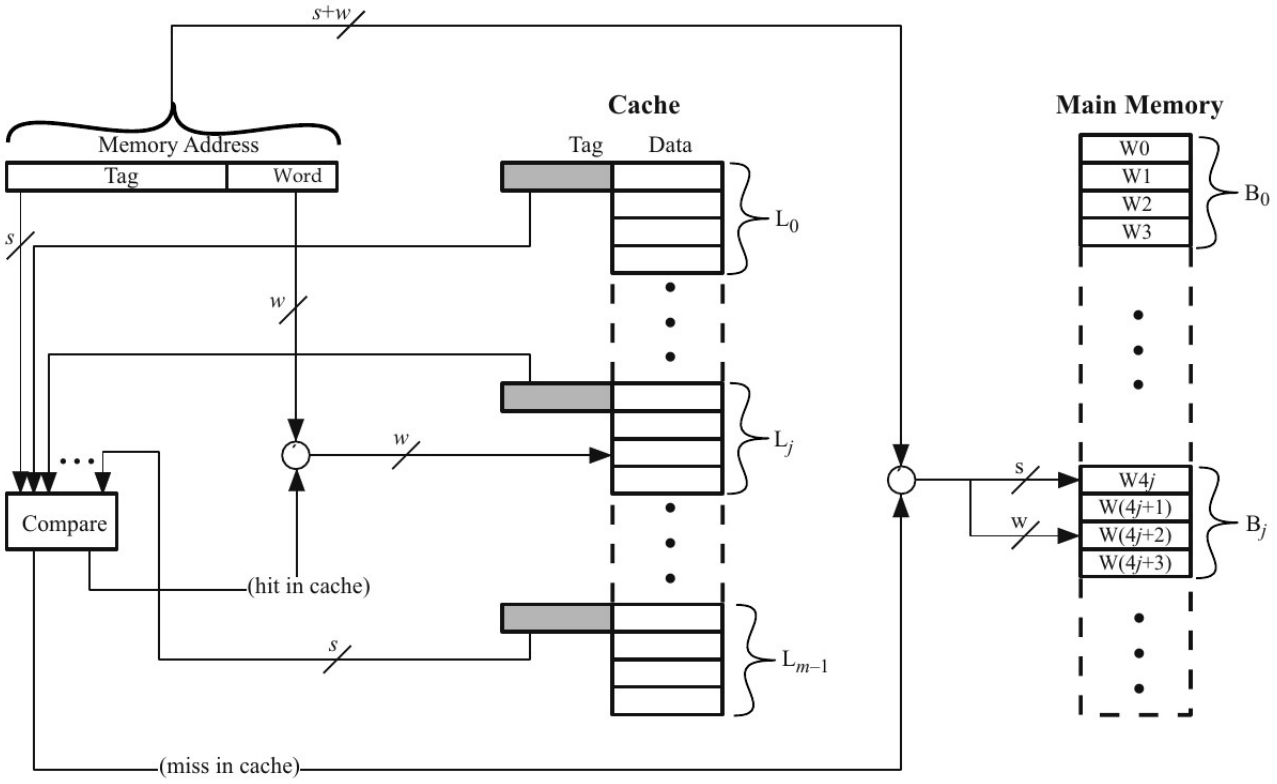



Fig. 11: Associative Mapping [Hwang 93], m = C

Associative memory access is very complex and expensive, hence an intermediate form of cache memory was also developed, called *set-associative* caching. In this case, the location of a block in the cache is not fixed as in direct caching, but also no longer completely free as in associative caching. However, the cache is divided into $v = 2^d$ sets, each containing exactly $k = C/v = 2^{r-d}$ lines (the first set contains the first *k* lines, the second the next k, etc.). Each block will now be allowed to use exactly one of these sets. In particular, when we look at the first *s* bits of an address, which indicate the block number, the last *d* of these bits will identify the set within which we may write the block. Since all the blocks that may be in a set contain the same last *d* bits, it suffices that the tag has length *s* - *d*. In this case, one speaks of a *k-way set-associative* cache.

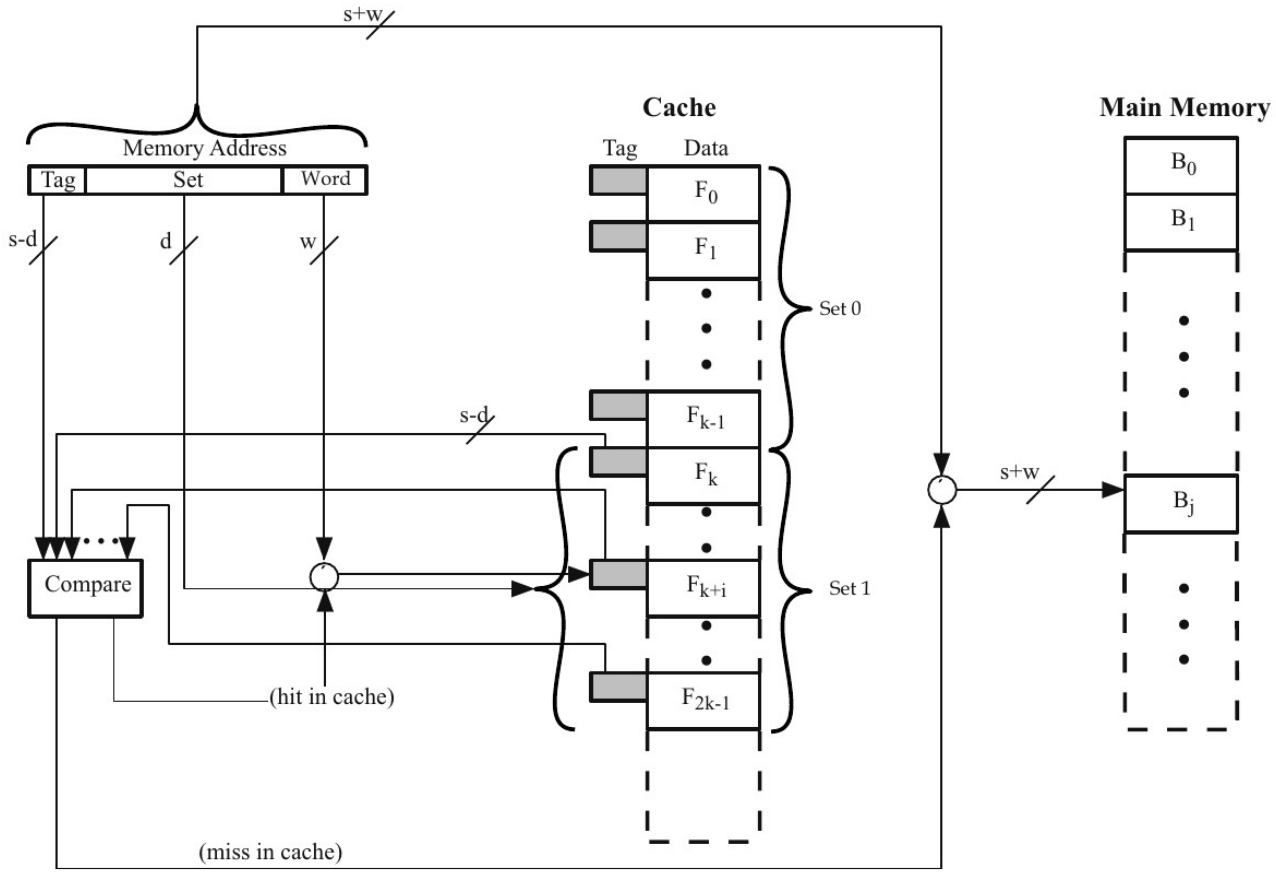



Fig. 12: Set Associative Mapping [Stallings 03].

Now when a particular word is looked up, the controller will first determine the set by looking at the d middle bits. Next, the tag of the address is compared to the *k* tags in the set to determine if the requested word is in the cache. When *k = 1* this method coincides with direct caching, while *k = C* indicates that we are dealing with associative caching. This intermediate solution is less expensive, since fewer tags need to be compared simultaneously. In addition, this intermediate solution also solves the problem of trashing to a large extent. When *k = 4*, one already has to repeatedly address 5 or more blocks that map to the same set. On the other hand, we also notice that the blocks that map onto the same set are closer together in main memory. In practice, *k* is usually very small, for example 2, 4, 8, etc. Such small values are already sufficient to avoid most cases of trashing. This is illustrated by Figure 13 for the PowerPC G3/4 that uses an 8-way associative cache (of 2 × 32 Kbyte).

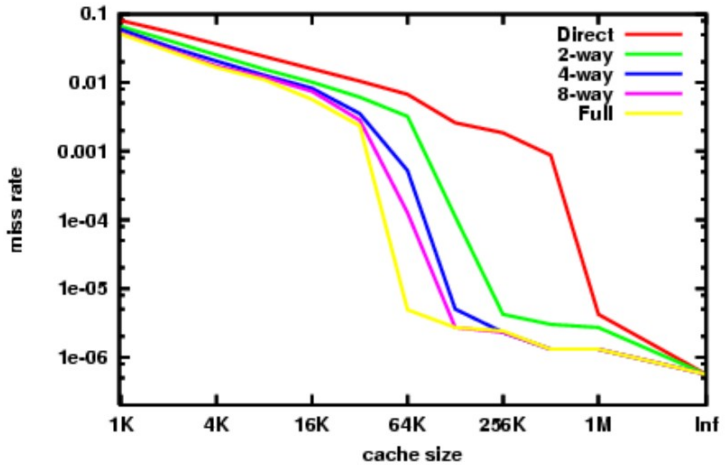



Fig. 13: Miss rate with set-associative caching in a PowerPC G3/4

## 2.3 Replacement algorithms

Unless one uses direct mapping, a choice always exists when we place a new block in cache memory. The algorithm we use to make this choice is called the *replacement* algorithm. Ideally, we always want to remove that block that we are least likely to call on again in the near future. Thus, in the context of the reference of locality property, the obvious strategy is to use a *least recently used* (LRU) strategy. This algorithm specifies that we choose from the set the block that has not been requested by the processor for the longest time. Of course, it requires some effort to keep track of when we called which blocks and in each set (in the case of set-associative caching). An obvious implementation of LRU, is to use a stack. When data is requested from a particular block within a set, we remove this block number from the stack and place it at the top of the stack. This stack requires $k \log_2 k$ bits, as there are $k$ elements in the set that we each in turn identify with $\log_2 k$ bits and this for each set.

A simpler solution is the *first in first out* (FIFO) method, where we always remove the oldest block. We can easily implement this with a circular buffer, requiring only $\log_2 k$ bits, per set. In case of FIFO, we assume that the oldest blocks are also the least recently used.

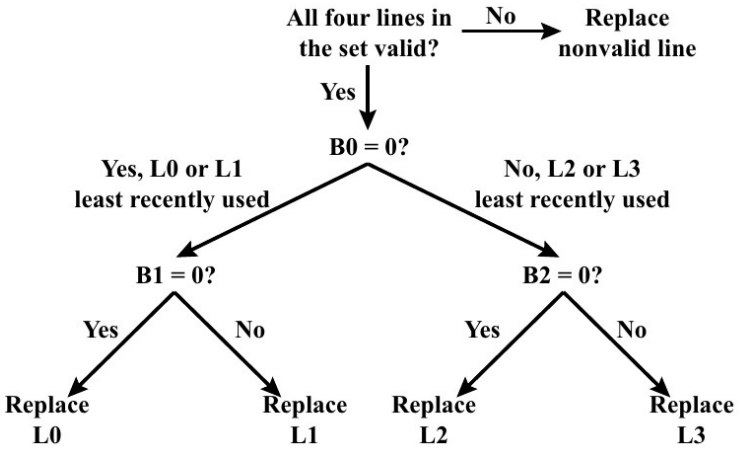



Fig. 14: Pseudo LRU cache replacement strategy on the Intel 40486 on-chip cache [Stallings 03].

In addition, pseudo LRU algorithms are sometimes used. Two important variants of this are the *tree-based* (TB) and *most recently used* (MRU) techniques. In case of the TB, a list of $k - 1$ bits is maintained: $B0, B1, \ldots, Bk - 2$. The first bit indicates whether it is the first $k/2$ lines or the last $k/2$ lines that are the least recently used. The second bit indicates which half of the first $k/2$ lines have been used least recently, while the third bit does this for the last $k/2$ lines, etc (see Figure 14). When we wish to write a new block into a set, we go through the tree to determine the line number. As we go through the tree, we also set the corresponding bit each time so that the branch we don't take is marked as the least recently used (i.e., we switch the bit). When a cache hit occurs, we also traverse the tree towards the requested cache location, adjusting the bits along the way if necessary. One can show that this algorithm will never delete one of the $\log_2 k = r - d$ most recently used blocks (or lines) (for $k \geq 2$). The Intel 80486 on-chip cache used the TB pseudo LRU algorithm with $k = 4$.

The MRU variant will keep a list of $k$ bits. When a hit happens on line i we set the $i$-th bit equal to 1 (unless it was already equal to 1). Now when we wish to insert a new block into the set, we find a block for which the corresponding bit is equal to 0 and replace this block (setting its bit to 1). When we replace the last 0 with a 1, we set all other bits back to zero. Note that we never overwrite the most recently used block, but possibly the second most recently used one. When we fill in the last 0 while inserting a new block, we certainly overwrite the least recently used block. For $k = 2$, both pseudo LRU algorithms coincide with LRU.

## 3 Memory updates: write policy

In addition to reading data from the cache, we naturally write into the cache. As we noted earlier, the information present in the cache is also in main memory. So when we only update the data in the cache there is a discrepancy between both memories. This can cause problems since other components, such as I/O equipment, can access this data without relying on the processor (via *direct memory access* (DMA)).

There are generally two techniques: *write through* and *write back*. In the first, simplest but also least efficient case, every update in the cache will be immediately applied to the main memory. This causes extra traffic on the bus when certain data is updated frequently. With write back, when changing the data in the cache, one will put an UPDATE bit, to indicate that this information is different from main memory. When the block is finally removed from the cache, it will be checked whether the UPDATE bit is present, if so then the main memory will be updated. Additional complications can arise when there are multiple processors each containing a cache with possibly the same information (see below).

## 4 Number of caches in today's systems

Contemporary computer systems use multiple caches. For example, (per core) there is usually a cache of limited size (e.g., 32 Kbyte) provided on the processor's chip, called the L1 cache. If the L1 cache does not contain the requested data, then the L2 cache is consulted. The L2 cache is usually a bit larger (e.g. 256 Kbyte), but slower and each core often has its own L2 cache. Finally, there is the L3 cache which is mostly shared between the different cores. The L3 cache contains several Mbyte of data, but is also the slowest of all caches.

| | |
|---|---|
| $L1$ CACHE hit | $\approx 4$ cycles |
| $L2$ CACHE hit | $\approx 10$ cycles |
| $L3$ CACHE hit | line unshared $\approx 40$ cycles |
| $L3$ CACHE hit | shared line in another core $\approx 65$ cycles |
| $L3$ CACHE hit | modified in another core $\approx 75$ cycles |
| Dram | $\approx 60 - 100$ ns |

Tab. 1: Intel Core i7 Xeon 5500 Series Data Source Latency.

When multiple caches are used, the question is whether the same data can/may be in only one or in multiple caches. With an inclusive cache, the contents of the *L1* cache will always be a subset of the *L2* cache (and analogous for the *L2* and *L3* caches). However, exclusive caches do not allow data to reside in different caches at the same time and a hit in the *L2* cache will in this case cause the line from the *L2* cache to be swapped with a line from the *L1* cache.

In practice, people also use a separate cache for instructions and data, which is referred to as a *split* cache. The disadvantage is that part of the cache capacity is lost. Sometimes there is a greater need for data, while at other times instructions are used most of the time. However, there are also strong advantages: one can retrieve information from both caches simultaneously. This is very efficient when the processor does *prefetching*, which indicates that the processor already tries to load part of the future instructions and data into the registers to reduce memory access time.

In the Virtual Memory chapter, we will see that contemporary processors are also equipped with TLB caches that are intended to quickly convert logical into physical memory addresses. Finally, for information, Table 1 shows the approximate times required to access the various caches on an Intel Core i7 machine. The difference between the three cases of an *L3* cache hit will become clear when we discuss the MESI protocol (further in the course).

## 5 Cache benchmarking

In this section, we discuss a simple test to discover the different properties of the cache system on your machine. The idea is to pass a number of arrays of integers, with $2^{CMIN}$ to $2^{CMAX}$ entries, a number of times each. During the first set of passes, all elements are requested. During the second sequence, we only request elements 2, 4, 6, etc. In general, at the i-th sequence of passages we read only entries $2^i$ , $2\ 2^i$ , $3\ 2^i$ , etc. We refer to the step size of a sequence of passages as the stride. By plotting the average access time as a function of the stride value for each array size we obtain a plot as shown in Figure 15. This figure is somewhat idealized, during the exercises a more realistic plot will be viewed.

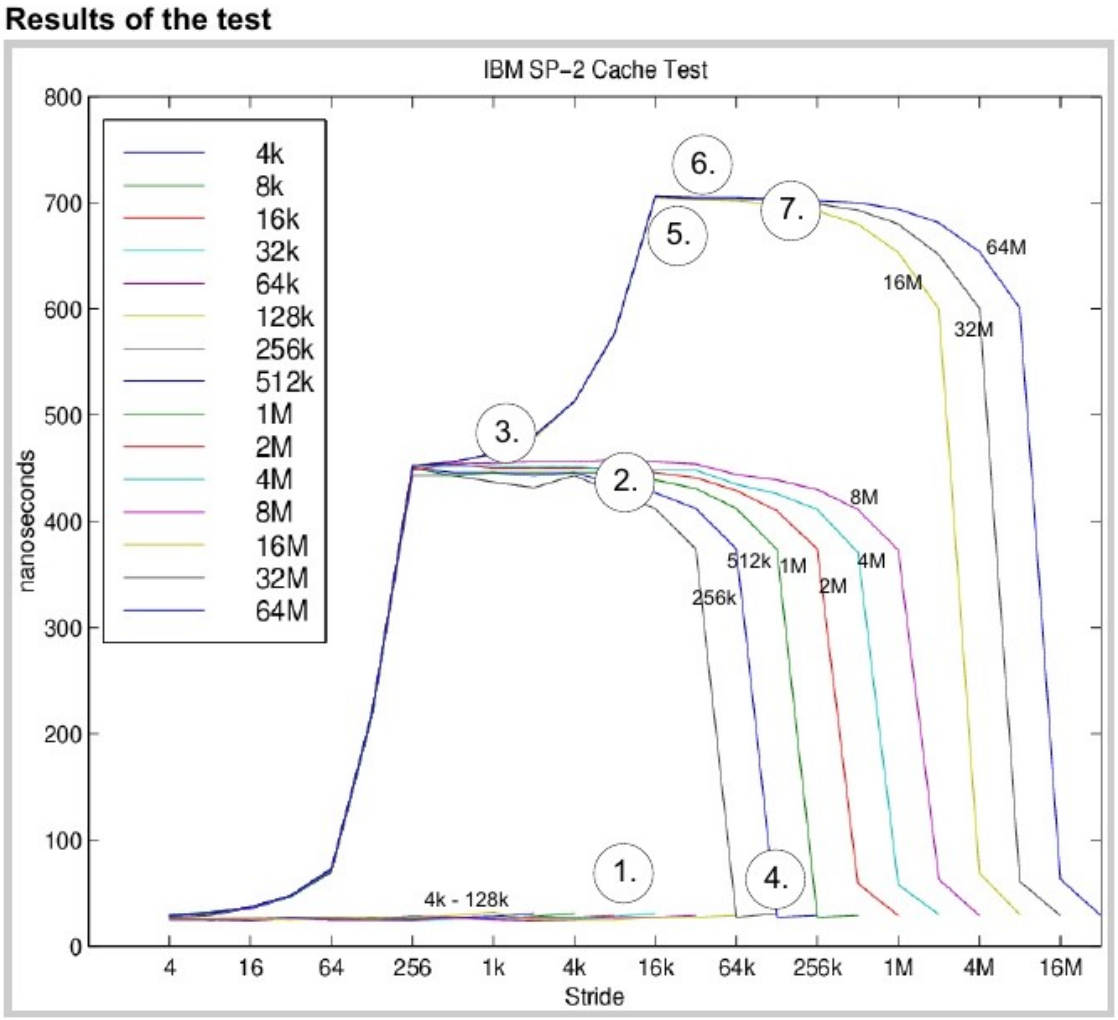



Fig. 15: Results of the benchmarking test.

We can learn a lot about our cache from this plot (the replacement policy you may assume LRU or FIFO):

1. **The size of the *L1* cache is 128 Kbyte**. Namely, for the arrays with a size of 128K or less, the access time is around 25 nsec. On the first pass, all elements are put in the cache, without being removed by subsequent passes. Therefore, there are no more misses. From 256K onwards, misses do occur as the average access time increases significantly. This indicates that the array no longer fits in the cache. For an array of 256K, the second half of the array will always overwrite the first half.

2. The additional computation time due to an *L1* cache miss is approximately 425 nanoseconds.

3. **A single cache line contains 256 bytes**. We can see this most easily from the results of arrays at least 256K in size. When the stride is half the size of a single cache line, there is at least one hit for every two words. Thus the access time is halfway between 25 and 450 nsec. If the stride is a quarter then there are at least three hits per miss. With a stride of 256 bytes there are only misses, each word therefore belongs to a different line.

4. **The *L1* cache is 4-way associative.** When the stride is almost as large as the array itself, then during a single pass almost no elements are read. For the array of 256K, a stride of 128K will result in reading only two elements, which are exactly 128K apart. Consequently, these two elements are always mapped to the same set (since the $r + w$ rightmost bits are identical). In the figure we see that these two elements do always result in hits, hence the associativity of the cache must be at least two. Since the sets contain at least two elements, the four elements of the 256K array, which we read at a 64K stride, are also mapped to the same set (because their last $r - 1 + w$ rightmost bits are identical). Again, we see that these four elements mostly result in hits, so a set must contain at least four elements. We can repeat this reasoning, up to the point where suddenly cache misses do occur most of the time, indicating that a single set can no longer contain all the elements to be read (assuming a not too stupid replacement algorithm is used). In this example, this happens for a stride of 32K, which corresponds to 8 elements. This allows us to conclude that the associativity of the *L1* cache should be 4.

5. **There is no *L2* cache**. The plot contains only 3 levels: an *L1* hit (25 nsec), an *L1* miss with a hit in the TLB cache (450 nsec, see Chapter Virtual Memory for more info on the TLB cache) and an *L1* miss with a miss in the TLB cache (700 nsec). If there was an additional level, it would be an *L2* cache.

6. **The TLB page size is 16K, the TLB cache contains 512 entries and is 4-way associative.** The transition from level two to level three occurs at a 16K stride. For smaller strides, the translation from the virtual address to the physical address will often be able to go through the TLB cache (because multiple elements are on the same page). Since the 8M array does not yet seem to cause TLB misses, the TLB cache must contain at least 512 entries, since 512 times 16K equals 8M. For the 16M array, however, there are misses, from which we may conclude that the TLB cache in all likelihood does not contain 1024 entries. Regarding the associativity, we observe the same behavior for at large arrays and strides as for the 256K array, indicating that the TLB cache is also 4-way associative. Should the TLB cache have a larger associativity, the times would first fall back to 450ns, because the TLB cache can contain the elements to be read in a single set, while this is not the case for the *L1* cache.

7. The additional computation time because of a TLB miss is 250 nsec.